Evaluation Harness for Customer Support Reply Generation

1. Task and Motivation

This project evaluates AI-generated customer-support replies. Given a customer message and a company policy, the system generates a reply that should answer the customer accurately while following the policy.

Exact-match evaluation is not appropriate because there is no single correct wording for a good customer-support response. Two replies can use different words while both being correct, complete, polite, and clear. Conversely, a fluent response can still violate company policy or invent information. Therefore, this project uses a quality-based evaluation framework rather than string matching.

The evaluation uses four criteria: correctness, completeness, politeness, and clarity. The company policy is treated as the source of truth.

2. Golden Dataset and Labelling

The golden dataset contains **150 customer-support scenarios**, covering refunds, returns, cancellations, shipping, late delivery, damaged or incorrect products, payments, discounts, subscriptions, warranties, accounts, passwords, privacy, and cases where the available information is insufficient.

The dataset contains:

- 100 development items
- 50 test items

Each item includes a customer message, company policy, candidate answer, and reference labels from two independent labelers.

The labelling guide defines:

- **5 – Excellent:** Correct, complete, polite, and clear.
- **4 – Good:** Mostly correct and useful, with only a minor omission or issue.
- **3 – Acceptable:** Useful but has a noticeable problem without a major false claim.
- **2 – Poor:** Contains a significant issue or misses important information.
- **1 – Very Poor:** Substantially incorrect, unusable, contradictory, or contains important invented information.

The two labelers agreed on **92 of 150 items (61.33%)**, with **58 disagreements**. Cohen's kappa was **0.447**, indicating moderate agreement.

For judge-reliability analysis, the 92 items where both labelers agreed were used as the human reference.

3. Evaluation Harness

The harness runs the complete evaluation from the dataset and saves one structured result per item.

The evaluation pipeline is:

Customer message + company policy

↓

Qwen2.5 7B

↓

Generated reply

↓

LLM judge

↓

Score and reason

The system uses **Qwen2.5 7B**, executed locally through Ollama.

The system prompt instructs the model to follow the policy, avoid unsupported claims, answer the customer's actual question, and remain clear and concise.

The judge receives the customer message, policy, and generated answer. It returns a structured JSON object containing a score from 1 to 5 and a short explanation.

Each result records the item ID, split, input information, generated answer, judge score, reason, generation latency, judge latency, total latency, and estimated cost.

4. LLM Judge Reliability

The initial v1 judge evaluation was compared against the reference labels from the 92 items where both human labelers agreed.

Measure	**v1**	**v2**
Resolved human items	92	92
Judge/reference agreements	38	43
Judge/reference disagreements	54	49
Exact agreement	41.30%	46.74%
Cohen's κ	0.178	0.202

The v1 judge produced 2 scores of 3, 48 scores of 4, and 42 scores of 5 on the resolved items.

With v2, the distribution changed to 3 scores of 3, 39 scores of 4, and 50 scores of 5.

The v2 prompt improved exact agreement from **41.30% to 46.74%** and Cohen's kappa from **0.178 to 0.202**.

The improvement is modest, but it shows that clearer policy-grounding and completeness instructions changed the judge's behaviour in the intended direction.

5. Position Bias

Position bias was tested by presenting two answers to the judge and then repeating the evaluation with their positions swapped. Scores were mapped back to the original answers so that the effect of answer position could be measured.

Out of 150 items:

- **54 items (36.00%)** produced unchanged scores.
- **96 items (64.00%)** changed after the answers were swapped.
- The **average absolute score change was 0.480 points**.

The result shows that answer order can influence the judge's evaluation. Therefore, a single judge evaluation should not automatically be assumed to be position-invariant.

6. Verbosity Bias

Verbosity bias was tested by taking each generated answer and appending irrelevant, repetitive customer-service text.

The results were:

Result after padding	Items	Percentage
Unchanged	67	44.67%
Score increased	3	2.00%
Score decreased	80	53.33%
Average absolute change	—	0.660

Only **2.00%** of answers received a higher score after irrelevant padding, while **53.33%** received a lower score.

This provides little evidence that the judge generally rewards unnecessary verbosity.

This experiment specifically measures sensitivity to irrelevant additional text. It does not determine whether useful additional information would improve an answer.

7. Prompt Improvement and System Score History

The project initially used `system_v1.txt`. A second version, `system_v2.txt`, was created with stronger instructions to:

- Answer every part of the customer's question.
- Avoid assumptions and unsupported claims.
- Avoid reinterpreting policy.
- Perform a final policy-support check.
- Avoid unnecessary repetition.

The same 50-item test set was used for both versions.

Measure	v1	v2
Test items	50	50
Mean score	4.480	4.600
Score 4	26	20
Score 5	24	30
Scores changed	—	10
Scores unchanged	—	40

The mean test-set score increased from **4.480 to 4.600 (+0.120)**.

Ten of the 50 test items changed score, while 40 remained unchanged. The v2 results therefore show an improvement on this test set.

8. System Performance and Cost

The system was evaluated locally using Qwen2.5 7B through Ollama, resulting in no direct API cost.

For the v1 run:

- **150 requests**
- **1,056.16 seconds** total system generation time
- **7.04 seconds/request** average generation latency
- **1,244.72 seconds** total judge time
- **8.30 seconds/item** average judge latency
- **15.34 seconds/item** average complete harness latency
- **300 LLM calls**
- **$0 direct API cost per 1,000 requests**

At 100 times the current workload, or **15,000 requests**, the observed v1 throughput would require approximately:

- **29.34 hours** of system generation time
- **34.58 hours** of judge time

For v2, generation latency increased to **9.47 seconds/request**, while judge latency was **12.05 seconds/request**, resulting in **21.52 seconds/item** for the complete harness.

At the same 15,000-request scale, v2 generation would require approximately **39.46 hours**.

The cost is $0 in terms of external API charges because the model runs locally. Actual electricity or hardware operating costs were not estimated.

9. Conclusion

This project demonstrates a complete evaluation workflow for a customer-support generation system rather than relying on a single quality score.

The human-labelling process produced **61.33% agreement and Cohen's** $\kappa = 0.447$, showing that the evaluation task itself contains some ambiguity.

The v1 LLM judge achieved **41.30% agreement** with the resolved human labels, while v2 increased this to **46.74%**.

The reliability experiments also exposed important judge behaviours. Position changes affected scores in **64.00%** of cases, while irrelevant verbosity caused score decreases in **53.33%** of cases.

The prompt improvement experiment increased the test-set mean from **4.480 to 4.600**, while also increasing inference latency.

Local execution resulted in **zero direct API cost**, but the system requires significant computation time at larger scale.

Overall, the project shows that an evaluation harness must itself be evaluated. Reliable measurement requires carefully defined criteria, independent labelling, structured outputs, judge-reliability testing, bias checks, and transparent cost and latency measurements.